第 4 章作业题解

题号：4.1，4.3，4.5，4.6，4.7，4.10，4.11，4.16，4.18，4.20，4.21，4.25，4.27

4.1

题目给定指令：

```
and rd, rs1, rs2
```

其语义为：

```
Reg[rd] = Reg[rs1] AND Reg[rs2]
```

这是一条典型的 R 型算术逻辑指令。

4.1.1 控制单元产生哪些控制信号？

`and` 指令只需要从寄存器堆读取两个源操作数，然后通过 ALU 执行 AND 操作，最后将结果写回目标寄存器 `rd`。

它不访问数据存储器，也不是分支或跳转指令。

控制信号如下：

控制信号	取值	说明
RegWrite	1	需要把 ALU 结果写回寄存器
ALUSrc	0	ALU 第二个输入来自寄存器 `rs2`
MemRead	0	不读数据存储器
MemWrite	0	不写数据存储器
MemtoReg	0	写回数据来自 ALU，而不是数据存储器
Branch	0	不是分支指令
ALUOp	R-type	由 funct 字段进一步确定为 AND

也可以写成：

```
RegWrite = 1
ALUSrc   = 0
MemRead  = 0
MemWrite = 0
MemtoReg = 0
Branch   = 0
ALUOp    = R-type
```

最终 ALU 控制单元根据 `funct` 字段产生：

```
ALUControl = AND
```

4.1.2 该指令会用到哪些功能部件?

该指令会用到以下功能部件：

1. PC
   用来给出当前指令地址。
2. 指令存储器
   根据 PC 取出 `and` 指令。
3. 寄存器堆
   读取 `rs1` 和 `rs2`，并在最后写回 `rd`。
4. ALU
   执行按位 AND 运算。
5. ALU 控制单元
   根据主控制信号和指令 funct 字段确定 ALU 执行 AND。
6. PC + 4 加法器
   计算下一条顺序执行指令的地址。
7. PC 多路选择器
   选择 `PC + 4` 作为下一条指令地址。
8. 写回多路选择器
   选择 ALU 结果写回寄存器堆。

4.1.3 哪些功能部件产生了输出但输出未被使用？哪些不产生有效输出?

对于 `and` 指令，部分功能部件虽然会产生输出，但这些输出不会被本指令使用。

产生输出但未被使用的部件：

1. 立即数扩展单元
   它可能会根据指令中的某些位产生扩展结果，但 R 型指令不需要立即数。
2. 分支目标地址加法器
   即使计算出某个地址，也不会被使用，因为 `and` 不是分支指令。
3. ALU 的 Zero 输出
   ALU 会产生 Zero 信号，但 `and` 指令不用它来决定 PC。
4. 数据存储器输出
   数据存储器没有被读取，所以其输出无意义或不被使用。

不产生有效输出的部件：

1. 数据存储器读端口
   因为 `MemRead = 0`，所以不会产生有效读数据。
2. 数据存储器写操作
   因为 `MemWrite = 0`，所以不会写入数据。

4.3

题目给出的指令组合为：

指令类型	比例
R-type	24%
I-type, non-lw	28%
Load	25%
Store	10%
Branch	11%

指令类型	比例
Jump	2%

4.3.1 多少指令会用到数据存储器?

数据存储器只被 Load 和 Store 指令使用。

```
Load  = 25%
Store = 10%
```

所以:

```
使用数据存储器的比例 = 25% + 10% = 35%
```

答案:

```
35%
```

4.3.2 多少指令会用到指令存储器?

所有指令都必须经过取指阶段，因此所有指令都需要使用指令存储器。

```
使用指令存储器的比例 = 100%
```

答案:

```
100%
```

4.3.3 多少指令会用到符号扩展?

需要立即数字段的指令会使用符号扩展。

需要符号扩展的指令包括:

```
I-type, non-lw
Load
Store
Branch
Jump
```

计算:

```
28% + 25% + 10% + 11% + 2% = 76%
```

答案:

```
76%
```

4.3.4 不需要符号扩展输出的时钟周期中，符号扩展部件在做什么？

符号扩展部件是组合逻辑。只要输入端有指令位，它就会根据输入位持续产生一个扩展后的输出。

但是对于不需要符号扩展的指令，比如 R 型指令：

```
符号扩展部件仍然会产生输出，
只是该输出不会被后续数据通路选择，
也不会影响处理器状态。
```

因此，它并不是停止工作，而是：

```
输出被忽略。
```

4.5

题目给定指令字：

```
0x00c6ba23
```

将其转换为二进制并按 RISC-V S 型指令格式解析：

```
0x00c6ba23
= 0000000 01100 01101 011 10100 0100011
```

字段划分：

```
opcode     = 0100011 = 0x23
funct3     = 011
rs1        = 01101 = x13
rs2        = 01100 = x12
imm[11:5]  = 0000000
imm[4:0]   = 10100
imm        = 000000010100 = 20
```

因此该指令是：

```
sd x12, 20(x13)
```

语义为：

```
Mem[Reg[x13] + 20] = Reg[x12]
```

4.5.1 对该指令，ALU 控制单元的输入值是什么？

该指令是 Store 类型指令，需要 ALU 计算数据存储器地址：

```
地址 = Reg[x13] + 20
```

因此 ALU 要执行加法。

主控制器给 ALU 控制单元的信号为：

```
ALUOp = 00
```

表示：

```
ALU 执行加法
```

所以 ALU 控制单元输出为：

```
ALUControl = ADD
```

答案：

```
ALUOp = 00
ALUControl = ADD
```

4.5.2 该指令执行后，新的 PC 地址是多少？指出决定该值的路径

该指令不是分支，也不是跳转。

所以 PC 顺序执行：

```
PC' = PC + 4
```

路径如下：

```
PC
→ PC + 4 加法器
→ PC 多路选择器选择 PC + 4
→ 写入 PC
```

答案：

```
新的 PC 地址 = PC + 4
```

4.5.3 给出指令执行过程中每个多路选择器的输入和输出值

该指令为：

```
sd x12, 20(x13)
```

寄存器堆读出：

```
Read register 1 = x13
Read register 2 = x12

Read data 1 = Reg[x13]
Read data 2 = Reg[x12]
```

立即数：

```
Imm = 20
```

ALUSrc 多路选择器

输入：

```
输入 0 = Reg[x12]
输入 1 = 20
```

由于 Store 指令要用立即数作为地址偏移：

```
ALUSrc = 1
```

输出：

```
ALUSrc Mux 输出 = 20
```

MemtoReg 写回多路选择器

输入：

```
输入 0 = ALU result = Reg[x13] + 20
输入 1 = Data memory read data
```

但是 Store 指令不写寄存器：

```
RegWrite = 0
```

所以该多路选择器输出不会被使用。

答案：

```
MemtoReg 输出 = don't care
```

PC 多路选择器

输入：

```
输入 0 = PC + 4
输入 1 = 分支目标地址
```

因为该指令不是分支：

```
Branch = 0
PCSrc  = 0
```

输出：

```
PC Mux 输出 = PC + 4
```

4.5.4 ALU 和两个加法器的输入值是什么？

ALU 输入

ALU 用来计算数据存储器地址：

```
ALU 输入 1 = Reg[x13]
ALU 输入 2 = 20
ALU 操作   = ADD
ALU 输出   = Reg[x13] + 20
```

该输出作为数据存储器地址：

```
D-Mem Address = Reg[x13] + 20
```

PC + 4 加法器输入

```
输入 1 = PC
输入 2 = 4
输出   = PC + 4
```

分支目标地址加法器输入

虽然本指令不是分支，但该加法器作为组合逻辑仍可能产生输出：

```
输入 1 = PC
输入 2 = 立即数 20
输出   = PC + 20
```

但是该输出不会被 PC 多路选择器选择。

4.5.5 寄存器单元的所有输入值是什么？

寄存器堆输入为：

```
ReadReg1  = x13
ReadReg2  = x12
WriteReg  = x20
WriteData = don't care
RegWrite  = 0
```

说明：

```
WriteReg = x20 是指令 [11:7] 字段对应的寄存器号。
但是 Store 指令不写寄存器，所以这个字段不会真正使用。
```

寄存器堆输出为：

```
ReadData1 = Reg[x13]
ReadData2 = Reg[x12]
```

最终执行效果：

```
Mem[Reg[x13] + 20] = Reg[x12]
```

4.6

题目讨论将 `addi` 或 `andi` 这一类 I 型指令加入 CPU。

4.6.1 如果在图 4-21 的 CPU 中增加 I 型指令，需要增加哪些逻辑块？

以 `addi` 为例：

```
addi rd, rs1, imm
```

语义为：

```
Reg[rd] = Reg[rs1] + imm
```

已有数据通路中已经包含：

1. 寄存器堆读端口
   可以读取 `rs1` 。
2. 立即数扩展单元
   可以产生立即数 `imm` 。
3. ALUSrc 多路选择器
   可以选择 ALU 第二个输入来自寄存器还是立即数。
4. ALU
   可以执行加法或逻辑运算。
5. 写回通路
   可以把 ALU 结果写回寄存器堆。

因此对于 `addi`：

```
不需要增加新的主要数据通路功能块。
```

需要修改的是：

```
1. 主控制器需要识别 I 型算术/逻辑指令。
2. ALU 控制单元需要根据 I 型指令的 funct 字段产生对应 ALU 操作。
```

对于 `andi`：

```
andi rd, rs1, imm
```

语义为：

```
Reg[rd] = Reg[rs1] AND imm
```

同样使用已有 ALU 和立即数通路。

4.6.2 `addi` 指令的控制信号是什么？哪些是无关控制信号？

`addi` 需要执行：

```
Reg[rd] = Reg[rs1] + imm
```

控制信号如下：

控制信号	取值	说明
RegWrite	1	结果写回 rd
ALUSrc	1	ALU 第二输入来自立即数
MemRead	0	不读数据存储器
MemWrite	0	不写数据存储器
MemtoReg	0	写回 ALU 结果
Branch	0	不是分支指令
ALUOp	ADD	ALU 执行加法

写成控制信号形式：

```
RegWrite = 1
ALUSrc   = 1
MemRead  = 0
MemWrite = 0
MemtoReg = 0
Branch   = 0
ALUOp    = ADD
```

无关项说明：

数据存储器读出的数据是无关的，因为 `MemRead = 0`。
分支目标地址和 `Zero` 信号是无关的，因为 `Branch = 0`。

但是：

`MemtoReg` 不是无关项。

因为 `addi` 要写回寄存器，并且必须选择 ALU 结果，所以：

```
MemtoReg = 0
```

4.7

给定各部件延迟：

部件	延迟
I-Mem / D-Mem	250 ps
Register File	150 ps
Mux	25 ps
ALU	200 ps
Adder	150 ps
Single gate	5 ps
Register Read	30 ps
Register Setup	20 ps
Sign extend	50 ps

部件	延迟
Control	50 ps

4.7.1 R 型指令的延迟是多少?

R 型指令路径为:

```
PC clk-to-Q
→ 指令存储器
→ 寄存器堆读
→ ALUSrc Mux
→ ALU
→ MemtoReg Mux
→ 寄存器堆 Setup
```

计算:

```
30 + 250 + 150 + 25 + 200 + 25 + 20 = 700 ps
```

答案:

```
R 型指令延迟 = 700 ps
```

4.7.2 `lw` 指令的延迟是多少?

`lw` 指令路径为:

```
PC clk-to-Q
→ 指令存储器
→ 寄存器堆读
→ ALUSrc Mux
→ ALU 计算地址
→ 数据存储器读
→ MemtoReg Mux
→ 寄存器堆 Setup
```

计算:

```
30 + 250 + 150 + 25 + 200 + 250 + 25 + 20 = 950 ps
```

答案:

```
lw 延迟 = 950 ps
```

4.7.3 `sw` 指令的延迟是多少?

`sw` 指令路径为:

```
PC clk-to-Q
→ 指令存储器
→ 寄存器堆读
→ ALUSrc Mux
→ ALU 计算地址
→ 数据存储器写
```

计算：

```
30 + 250 + 150 + 25 + 200 + 250 = 905 ps
```

答案：

```
sw 延迟 = 905 ps
```

4.7.4 `beq` 指令的延迟是多少？

`beq` 需要比较两个寄存器，并根据 Zero 和 Branch 决定 PC。

路径为：

```
PC clk-to-Q
→ 指令存储器
→ 寄存器堆读
→ ALUSrc Mux
→ ALU 比较
→ AND gate
→ PC Mux
→ PC Setup
```

计算：

```
30 + 250 + 150 + 25 + 200 + 5 + 25 + 20 = 705 ps
```

答案：

```
beq 延迟 = 705 ps
```

4.7.5 算术逻辑指令或 I 型移位指令的延迟是多少？

这类指令路径与 R 型类似：

```
PC clk-to-Q
→ 指令存储器
→ 寄存器堆读
→ ALUSrc Mux
→ ALU
→ MemtoReg Mux
→ 寄存器堆 Setup
```

计算：

```
30 + 250 + 150 + 25 + 200 + 25 + 20 = 700 ps
```

答案：

```
算术逻辑或 I 型非 load 指令延迟 = 700 ps
```

4.7.6 该 CPU 的最小时钟周期是多少?

单周期 CPU 的时钟周期必须大于等于最慢指令延迟。

各指令延迟：

指令	延迟
R-type	700 ps
lw	950 ps
sw	905 ps
beq	705 ps
I-type 非 load	700 ps

最长的是 `lw`：

```
最小时钟周期 = 950 ps
```

答案：

```
950 ps
```

4.10

题目条件：

```
寄存器数量从 32 个增加到 64 个。
lw 和 sw 指令数量减少 12%。
寄存器堆延迟从 150 ps 增加到 160 ps。
寄存器堆开销从 200 增加到 400。
```

采用练习 4.8 的指令比例：

指令类型	比例
R-type / I-type non-lw	52%
lw	25%
sw	11%
beq	12%

4.10.1 改进后的性能加速比是多少?

原 CPU 周期由 `lw` 决定：

```
T_old = 950 ps
```

寄存器堆延迟增加 10 ps，因此新的 `lw` 延迟为：

```
T_new = 30 + 250 + 160 + 25 + 200 + 250 + 25 + 20
      = 960 ps
```

`lw` 和 `sw` 比例为：

```
25% + 11% = 36%
```

这些指令数量减少 12%，因此总指令条数减少：

```
36% × 12% = 0.36 × 0.12 = 0.0432 = 4.32%
```

新指令条数为：

```
I_new = 0.9568 I_old
```

原执行时间：

```
Time_old = I_old × 950
```

新执行时间：

```
Time_new = 0.9568 I_old × 960
```

加速比：

```
Speedup = Time_old / Time_new
        = 950 / (0.9568 × 960)
        ≈ 1.034
```

答案：

```
加速比约为 1.034
即性能提高约 3.4%
```

4.10.2 比较性能变化和开销变化

原部件开销：

```
I-Mem       = 1000
Register   = 200
Mux         = 10
ALU         = 100
Adder       = 30
D-Mem       = 2000
Single Reg  = 5
Sign extend = 100
Sign gate   = 1
Control     = 500
```

原总开销：

```
1000 + 200 + 10 + 100 + 30 + 2000 + 5 + 100 + 1 + 500 = 3946
```

寄存器堆开销从 200 增加到 400，所以新总开销：

```
3946 - 200 + 400 = 4146
```

开销比：

```
Cost ratio = 4146 / 3946 ≈ 1.051
```

即开销增加约：

```
5.1%
```

性能提高为：

```
Speedup ≈ 1.034
```

即性能提高约：

```
3.4%
```

性能/开销比变化：

```
性能/开销比 = 1.034 / 1.051 ≈ 0.984
```

说明：

```
性能/开销比下降约 1.6%。
```

结论：

```
虽然性能提高了，但硬件开销增加更多，因此从性能/开销比看不一定划算。
```

4.10.3 什么情况下增加更多寄存器有意义？

设 `lw/sw` 减少比例为 `r`。

因为 `lw/sw` 原本占：

```
36%
```

所以新指令条数因子为：

```
1 - 0.36r
```

如果只要求性能提高，需要：

```
(1 - 0.36r) × 960 < 950
```

解得：

```
r > 0.0289
```

也就是说：

```
只要 lw/sw 减少超过约 2.9%，性能就能提高。
```

但如果考虑性能/开销比，需要：

```
Speedup > Cost ratio
```

即：

```
950 / [960(1 - 0.36r)] > 4146 / 3946
```

解得大约：

```
r > 16.2%
```

结论：

```
当程序寄存器压力较大，增加寄存器能显著减少 load/store 指令时，
增加寄存器是有意义的。

如果程序中 load/store 减少不明显，
增加寄存器带来的周期变长和成本增加会抵消收益，
则增加更多寄存器没有意义。
```

4.11

新增指令：

```
lwi rd, rs1, rs2
```

语义：

```
Reg[rd] = Mem[Reg[rs1] + Reg[rs2]]
```

也就是用两个寄存器中的值相加作为访存地址。

4.11.1 该指令需要哪些新的功能块?

已有数据通路已经包括：

1. 寄存器堆两个读端口
   可以同时读取 `rs1` 和 `rs2`。
2. ALU
   可以执行加法，计算地址：

   ```
   Reg[rs1] + Reg[rs2]
   ```

3. 数据存储器
   可以根据 ALU 输出地址读数据。
4. 写回通路
   可以把数据存储器读出的数据写回 `rd`。

因此：

```
不需要增加新的主要功能块。
```

4.11.2 哪些已有功能块需要修改?

需要修改主控制器，使其识别 `lwi` 指令，并产生新的控制信号组合。

普通 `lw`：

```
地址 = Reg[rs1] + imm
```

`lwi`：

```
地址 = Reg[rs1] + Reg[rs2]
```

所以两者区别在于 ALU 第二输入来源。

对于 `lwi`：

```
ALUSrc = 0
```

因为第二个操作数来自寄存器 `rs2`，不是立即数。

控制信号为：

```
RegWrite = 1
MemRead  = 1
MemWrite = 0
MemtoReg = 1
ALUSrc   = 0
Branch   = 0
ALUOp    = ADD
```

4.11.3 该指令需要哪些新的数据通路?

基本不需要新的数据通路。

已有路径可以完成：

```
Reg[rs1] → ALU 输入 1
Reg[rs2] → ALU 输入 2
ALU 输出 → 数据存储器地址
数据存储器读数据 → 写回寄存器 rd
```

因此：

```
不需要新数据通路，只需要组合已有 R 型寄存器读路径和 lw 的访存写回路径。
```

4.11.4 控制器中需要哪些新的控制信号?

不需要新的控制信号，只需要为 `lwi` 添加新的控制信号取值组合。

`lwi` 控制信号：

控制信号	取值
RegWrite	1
MemRead	1

控制信号	取值
MemWrite	0
MemtoReg	1
ALUSrc	0
Branch	0
ALUOp	ADD

关键区别：

```
普通 lw:
    ALUSrc = 1

lwi:
    ALUSrc = 0
```

所以：

```
lwi 不需要新增控制信号，只需要修改控制器的译码逻辑。
```

4.16

给定流水级延迟：

阶段	延迟
IF	250 ps
ID	350 ps
EX	150 ps
MEM	300 ps
WB	200 ps

指令比例：

指令类型	比例
ALU/Logic	45%
Jump/Branch	20%
Load	20%
Store	15%

4.16.1 流水线处理器和非流水线处理器的时钟周期分别是多少？

流水线处理器的时钟周期由最慢流水级决定：

```
T_pipeline = max(250, 350, 150, 300, 200)
           = 350 ps
```

非流水线单周期处理器需要在一个周期内完成所有阶段：

```
T_nonpipeline = 250 + 350 + 150 + 300 + 200
              = 1250 ps
```

答案：

```
流水线处理器时钟周期      = 350 ps
非流水线处理器时钟周期    = 1250 ps
```

4.16.2 `lw` 指令在流水线和非流水线处理器中的总延迟是多少？

非流水线中，一条 `lw` 在一个长周期中完成：

```
lw 非流水线延迟 = 1250 ps
```

流水线中，一条 `lw` 需要经过 5 个流水级：

```
lw 流水线延迟 = 5 × 350 = 1750 ps
```

答案：

```
非流水线 lw 延迟 = 1250 ps
流水线 lw 延迟   = 1750 ps
```

说明：

```
流水线主要提高吞吐率，不一定降低单条指令延迟。
```

4.16.3 如果把一个流水级拆成两级，每级延迟为原来一半，应该拆哪一级？是否所有指令受益？

应该拆分最长的流水级：

```
ID = 350 ps
```

拆分后：

```
ID1 = 175 ps
ID2 = 175 ps
```

新的流水级延迟为：

```
IF  = 250 ps
ID1 = 175 ps
ID2 = 175 ps
EX  = 150 ps
MEM = 300 ps
WB  = 200 ps
```

新的时钟周期：

```
T_new = max(250, 175, 175, 150, 300, 200)
      = 300 ps
```

吞吐率提高：

```
350 / 300 ≈ 1.167
```

即：

```
约提高 16.7%
```

是否所有指令受益：

```
从吞吐率角度看，所有指令都受益，因为时钟周期变短。
```

但是单条指令延迟从：

```
5 × 350 = 1750 ps
```

变为：

```
6 × 300 = 1800 ps
```

因此：

```
单条指令延迟略微增加。
```

4.16.4 假设没有阻塞和冒险，数据存储器的利用率是多少？

数据存储器只被 Load 和 Store 使用。

```
Load  = 20%
Store = 15%
```

因此：

```
数据存储器利用率 = 20% + 15% = 35%
```

答案：

```
35%
```

4.16.5 假设没有阻塞和冒险，寄存器堆写寄存器端口的利用率是多少？

会写寄存器的指令有：

```
ALU/Logic
Load
```

比例为：

```
45% + 20% = 65%
```

答案：

```
寄存器堆写端口利用率 = 65%
```

4.18

初值：

```
$s0 = 11
$s1 = 22
```

代码：

```
addi $s0, $s1, 5
add  $s2, $s0, $s1
addi $s3, $s0, 15
```

流水线不处理数据冒险，也没有插入 NOP。

流水线时序分析

令：

```
I1 = addi $s0, $s1, 5
I2 = add  $s2, $s0, $s1
I3 = addi $s3, $s0, 15
```

流水线时序：

周期	I1	I2	I3
1	IF		
2	ID	IF	
3	EX	ID	IF
4	MEM	EX	ID
5	WB	MEM	EX
6		WB	MEM
7			WB

第一条指令真正写回 `$s0` 的时间是第 5 周期 WB：

```
$s0 = $s1 + 5 = 22 + 5 = 27
```

但是：

```
I2 在第 3 周期 ID 读寄存器
I3 在第 4 周期 ID 读寄存器
```

此时 I1 还没有写回，因此 I2 和 I3 读到的都是旧值：

```
$s0_old = 11
```

所以：

```
$s2 = 11 + 22 = 33
$s3 = 11 + 15 = 26
```

答案：

```
$s2 = 33
$s3 = 26
```

4.20

原代码：

```
addi $s0, $s1, 5
add  $s2, $s0, $s1
addi $s3, $s0, 15
add  $s4, $s2, $s1
```

要求在无数据冒险处理的流水线中正确执行，需要手动插入 NOP。

冒险分析

第一条指令：

```
addi $s0, $s1, 5
```

产生 `$s0` 。

后两条指令：

```
add  $s2, $s0, $s1
addi $s3, $s0, 15
```

都需要使用新的 `$s0` 。

在没有旁路、没有冒险检测的流水线中，消费者的 ID 阶段必须等到生产者 WB 阶段。

因此在第一条指令之后需要插入 2 个 NOP：

```
addi $s0, $s1, 5
nop
nop
add  $s2, $s0, $s1
```

然后：

```
add $s4, $s2, $s1
```

依赖于：

```
add $s2, $s0, $s1
```

中间已有一条独立指令：

```
addi $s3, $s0, 15
```

还需要再插入 1 个 NOP。

最终代码

```
addi $s0, $s1, 5
nop
nop
add  $s2, $s0, $s1
addi $s3, $s0, 15
nop
add  $s4, $s2, $s1
```

4.21

题目条件：

```
无旁路流水线：
    时钟周期 = 250 ps
    NOP 数量 = 0.4n

有旁路流水线：
    时钟周期 = 300 ps
    NOP 数量 = 0.05n
```

4.21.1 新流水线相对无旁路流水线的加速比是多少？

无旁路执行时间：

```
Time_old = (n + 0.4n) × 250
         = 1.4n × 250
         = 350n
```

有旁路执行时间：

```
Time_new = (n + 0.05n) × 300
         = 1.05n × 300
         = 315n
```

加速比：

```
Speedup = Time_old / Time_new
        = 350n / 315n
        ≈ 1.111
```

答案：

```
加速比约为 1.11
```

4.21.2 带旁路流水线中最多还能保留多少 NOP，才不会更慢？

设带旁路后还剩：

```
yn 条 NOP
```

则有旁路执行时间：

```
Time_new = (1 + y)n × 300
```

无旁路执行时间：

```
Time_old = 1.4n × 250
```

要求：

```
Time_new ≤ Time_old
```

代入：

```
(1 + y)n × 300 ≤ 1.4n × 250
```

消去 n：

```
(1 + y) × 300 ≤ 350
```

所以：

```
1 + y ≤ 350 / 300
1 + y ≤ 1.1667
y ≤ 0.1667
```

答案：

```
最多还能保留 0.1667n 条 NOP
即约为原始指令数的 16.7%
```

4.21.3 用 x 表示无旁路时 NOP 数量相对 n 的比例，答案如何表示？

设无旁路时：

```
NOP = xn
```

无旁路执行时间：

```
Time_old = (1 + x)n × 250
```

设有旁路后剩余 NOP 为：

```
yn
```

有旁路执行时间：

```
Time_new = (1 + y)n × 300
```

要求有旁路不更慢：

```
(1 + y)n × 300 ≤ (1 + x)n × 250
```

消去 n：

```
1 + y ≤ (250 / 300)(1 + x)
```

所以：

```
y ≤ (5/6)(1 + x) - 1
```

化简：

```
y ≤ (5x - 1) / 6
```

答案：

```
y_max = (5x - 1) / 6
```

如果：

```
x < 0.2
```

则：

```
y_max < 0
```

表示即使旁路消除所有 NOP，也不能弥补时钟周期变长的损失。

4.21.4 一个只用 0.075n 条 NOP 的程序，能否在带旁路流水线中更快?

无旁路执行时间：

```
Time_old = (1 + 0.075)n × 250
         = 1.075n × 250
         = 268.75n
```

带旁路的最好情况是假设所有 NOP 都被消除：

```
Time_new_best = n × 300
              = 300n
```

比较：

```
300n > 268.75n
```

所以：

```
不可能更快。
```

原因：

```
原程序中的 NOP 比例太少，只有 7.5%，不足以抵消时钟周期从 250 ps 增加到 300 ps 的损失。
```

4.21.5 至少要有多少 NOP，程序才可能在带旁路流水线中更快?

设无旁路时 NOP 比例为 x。

带旁路的最好情况：

$$\text{所有 NOP 都消除}$$

要求：

$$n \times 300 < (1 + x)n \times 250$$

消去 n：

$$300 < 250(1 + x)$$

解得：

$$1.2 < 1 + x$$
$$x > 0.2$$

答案：

$$\text{至少要超过 } 0.2n \text{ 条 NOP，}$$
$$\text{即超过原始指令数的 } 20\%,$$
$$\text{带旁路流水线才可能更快。}$$

如果刚好是 20%，两者执行时间相同；必须超过 20% 才能更快。

4.25

循环代码：

```
LOOP:
    ld   $s0, 0($s3)
    ld   $s1, 8($s3)
    add  $s2, $s0, $s1
    addi $s3, $s3, -16
    bnez $s2, LOOP
```

条件：

1. 使用完美分支预测，没有控制冒险停顿。
2. 没有延迟槽。
3. 流水线有完全旁路。
4. 分支在 EX 阶段解析。

4.25.1 画出前两次迭代的流水线执行图

记号：

```
ld0.1   = 第 1 次迭代的 ld $s0, 0($s3)
ld1.1   = 第 1 次迭代的 ld $s1, 8($s3)
add.1   = 第 1 次迭代的 add
addi.1  = 第 1 次迭代的 addi
bnez.1  = 第 1 次迭代的 bnez

ld0.2   = 第 2 次迭代的 ld $s0, 0($s3)
ld1.2   = 第 2 次迭代的 ld $s1, 8($s3)
add.2   = 第 2 次迭代的 add
addi.2  = 第 2 次迭代的 addi
bnez.2  = 第 2 次迭代的 bnez
```

关键数据冒险：

```
ld   $s1, 8($s3)
add  $s2, $s0, $s1
```

这是典型 load-use 冒险。即使有完全旁路，load 的数据也要到 MEM 结束后才可用，因此紧随其后的使用者需要停顿 1 个周期。

流水线执行图：

指令	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C
ld0.1	IF	ID	EX	MEM	WB											
ld1.1		IF	ID	EX	MEM	WB										
add.1			IF	ID	ID(stall)	EX	MEM*	WB								
addi.1				IF	IF(stall)	ID	EX	MEM*	WB							
bubble					EX	MEM*	WB*									
bnez.1						IF	ID	EX	MEM*	WB*						
ld0.2							IF	ID	EX	MEM	WB					
ld1.2								IF	ID	EX	MEM	WB				
add.2									IF	ID	ID(stall)	EX	MEM*	WB		
addi.2										IF	IF(stall)	ID	EX	MEM*	WB	
bubble											EX	MEM*	WB*			
bnez.2												IF	ID	EX	MEM*	W

说明：

```
MEM*  表示该阶段不真正访问数据存储器。
WB*   表示该阶段不真正写寄存器。
stall 表示由于 load-use 冒险造成的停顿。
```

4.25.2 标出无用流水级，满流水时五级都做有用工作的频率是多少？

无用工作包括：

1. load-use 冒险插入的 bubble。
2. ALU 指令和分支指令的 MEM 阶段。
3. Store 和 Branch 等不写寄存器指令的 WB 阶段。
4. 停顿时 IF/ID 中重复保持的阶段。

题目要求观察范围：

从 addi 指令位于 IF 阶段的周期开始，
到 bnez 指令位于 IF 阶段的周期结束。

以第一次迭代为例：

addi.1 在 IF 的周期是 C4 和 C5
bnez.1 在 IF 的周期是 C6

观察 C4 到 C6：

C4：流水线尚未完全满载。
C5：发生 load-use 停顿，EX 阶段有 bubble。
C6：bubble 进入 MEM，MEM 阶段无有效工作。

因此该区间没有任何一个周期五个阶段都在执行有用工作。

答案：

五级都做有用工作的频率 = 0%

4.27

指令序列：

```
add $s3, $s1, $s0
lw  $s2, 4($s3)
lw  $s1, 0($s4)
or  $s2, $s3, $s2
sw  $s2, 0($s3)
```

依赖关系：

I1: add 产生 $s3
I2: lw 使用 $s3
I4: or 使用 $s3 和 $s2
I5: sw 使用 $s3 和 $s2

主要 RAW 相关：

I1 → I2: $s3
I1 → I4: $s3
I1 → I5: $s3
I2 → I4: $s2
I4 → I5: $s2

4.27.1 没有旁路或冒险检测时，插入 NOP 保证正确执行

无旁路、无冒险检测时，消费者必须等到生产者 WB 之后才能在 ID 阶段正确读寄存器。

原代码：

```
add $s3, $s1, $s0
lw  $s2, 4($s3)
lw  $s1, 0($s4)
or  $s2, $s3, $s2
sw  $s2, 0($s3)
```

插入 NOP 后：

```
add $s3, $s1, $s0
nop
nop
lw  $s2, 4($s3)
lw  $s1, 0($s4)
nop
or  $s2, $s3, $s2
nop
nop
sw  $s2, 0($s3)
```

解释：

1. `add → lw` 使用 `$s3`，需要 2 个 NOP。
2. `lw $s2 → or` 使用 `$s2`，中间已有一条 `lw $s1`，还需要 1 个 NOP。
3. `or → sw` 使用新的 `$s2` 作为存储数据，需要 2 个 NOP。

4.27.2 修改或重排代码，使 NOP 数量尽量少

可以把与 `$s3` 无关的指令：

```
lw $s1, 0($s4)
```

提前，用来填补一部分等待时间。

一种可行写法：

```
add $s3, $s1, $s0
lw  $s1, 0($s4)
nop
lw  $s2, 4($s3)
nop
nop
or  $s2, $s3, $s2
nop
nop
sw  $s2, 0($s3)
```

验证：

1. `add` 写回 `$s3` 后，`lw $s2, 4($s3)` 才在 ID 阶段读取 `$s3`。
2. `lw $s2` 写回后，`or` 才读取 `$s2`。
3. `or` 写回 `$s2` 后，`sw` 才读取新的 `$s2` 作为写入内存的数据。

这段代码仍需要 5 个 NOP。

原因：

> 可移动的独立指令太少，无法完全填满所有 `RAW` 冒险产生的空隙。

4.27.3 如果处理器有旁路，但忘记实现冒险检测单元，原始代码执行会发生什么？

原始代码：

```
add $s3, $s1, $s0
lw  $s2, 4($s3)
lw  $s1, 0($s4)
or  $s2, $s3, $s2
sw  $s2, 0($s3)
```

分析：

1. `add → lw` 的 `$s3` 相关
   `lw` 在 EX 阶段计算地址时需要 `$s3`，可以通过 EX/MEM → EX 旁路得到。
2. `lw $s2 → or` 的 `$s2` 相关
   两条指令之间隔了一条 `lw $s1`，所以当 `or` 到 EX 阶段时，`lw $s2` 的数据可以通过 MEM/WB → EX 旁路得到。
3. `or → sw` 的 `$s2` 相关
   `sw` 需要把 `$s2` 写入数据存储器。若旁路完整，包括对 store 写数据的旁路，则可以将 `or` 的结果旁路给 `sw`。

因此：

该原始序列没有紧邻 `load-use` 冒险。

所以：

即使没有冒险检测单元，只要旁路完整，该代码仍可以正确执行。

答案：

代码可以正确执行，不会因为缺少冒险检测而出错。

4.27.4 有旁路时，前 7 个周期中冒险检测单元和旁路单元产生哪些信号？

指令编号：

```
I1 = add $s3, $s1, $s0
I2 = lw  $s2, 4($s3)
I3 = lw  $s1, 0($s4)
I4 = or  $s2, $s3, $s2
I5 = sw  $s2, 0($s3)
```

前 7 个周期流水线状态：

周期	IF	ID	EX	MEM	WB
C1	I1				
C2	I2	I1			
C3	I3	I2	I1		
C4	I4	I3	I2	I1	
C5	I5	I4	I3	I2	I1
C6		I5	I4	I3	I2
C7			I5	I4	I3

旁路控制编码：

```
ForwardA = 00：ALU 输入 A 来自 ID/EX
ForwardA = 10：ALU 输入 A 来自 EX/MEM
ForwardA = 01：ALU 输入 A 来自 MEM/WB

ForwardB = 00：ALU 输入 B 来自 ID/EX
ForwardB = 10：ALU 输入 B 来自 EX/MEM
ForwardB = 01：ALU 输入 B 来自 MEM/WB
```

信号表：

周期	EX 阶段指令	冒险检测单元	ForwardA	ForwardB	说明
C1	无	无停顿	00	00	无 EX 指令
C2	无	无停顿	00	00	无 EX 指令
C3	I1 add	无停顿	00	00	不需要旁路
C4	I2 lw	无停顿	10	00	`lw` 地址计算需要 `$s3`，来自 I1 的 EX/MEM
C5	I3 lw	无停顿	00	00	使用 `$s4`，无相关
C6	I4 or	无停顿	00	01	`or` 的 `$s2` 来自 I2 的 MEM/WB
C7	I5 sw	无停顿	00	10	`sw` 的写数据 `$s2` 来自 I4 的 EX/MEM

冒险检测单元没有检测到紧邻 load-use 冒险，因此：

```
PCWrite    = 1
IF/IDWrite = 1
控制信号不清零
```

即：

```
不插入 bubble。
```

4.27.5 如果没有旁路，冒险检测单元需要哪些新的输入和输出信号？

原本的冒险检测单元通常只检测紧邻的 load-use 冒险：

```
ID/EX.MemRead = 1
且 ID/EX.Rd 等于 IF/ID.Rs1 或 IF/ID.Rs2
```

但是如果没有旁路，任何尚未 WB 的生产者都可能造成 RAW 冒险。

因此冒险检测单元必须检测：

```
当前 ID 阶段指令是否读取了 EX 阶段指令将写的寄存器；
当前 ID 阶段指令是否读取了 MEM 阶段指令将写的寄存器。
```

需要新增或使用的输入信号

需要从流水线寄存器送入冒险检测单元：

```
ID/EX.RegWrite
ID/EX.Rd

EX/MEM.RegWrite
EX/MEM.Rd

IF/ID.Rs1
IF/ID.Rs2
```

如果要避免对 I 型指令等产生误判，还需要知道当前指令是否真的使用某个源寄存器：

```
IF/ID.UsesRs1
IF/ID.UsesRs2
```

需要的输出信号

输出仍然是停顿控制信号：

```
PCWrite = 0
```

表示：

```
停止 PC 更新
```

```
IF/IDWrite = 0
```

表示：

```
冻结 IF/ID 流水线寄存器
```

```
ID/EX 控制信号清零
```

表示：

```
向流水线中插入 bubble
```

这些信号可能需要连续保持多个周期，因为没有旁路时，消费者必须等生产者到 WB 后才能正确读取寄存器。

用本题说明为什么需要这些信号

代码：

```
add $s3, $s1, $s0
lw  $s2, 4($s3)
lw  $s1, 0($s4)
or  $s2, $s3, $s2
sw  $s2, 0($s3)
```

例子 1：

```
add 产生 $s3，
紧随其后的 lw 要用 $s3 计算地址。
```

如果没有旁路，必须检测：

```
ID/EX.Rd == IF/ID.Rs1
```

或：

```
EX/MEM.Rd == IF/ID.Rs1
```

然后停顿。

例子 2：

```
lw $s2 产生 $s2，
or 要读取 $s2。
```

必须检测：

```
EX/MEM.Rd == IF/ID.Rs2
```

例子 3：

```
or 产生新的 $s2，
sw 需要读取 $s2 作为写入内存的数据。
```

因此 store 的数据寄存器也必须被视为源寄存器参与冒险检测。

结论：

```
没有旁路时，冒险检测单元必须检测所有尚未写回的 RAW 相关，
而不能只检测 load-use 冒险。
```